~\Desktop\C++学习资料\Untitled-1.cpp

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <atomic>
#include <future>
#include <functional>
#include <concepts>
#include <syncstream> // C++20: 解决cout乱序
#include <new>        // for std::hardware_destructive_interference_size

// ---------------------------------------------------------------------------
// 1. 无锁 MPMC 环形队列 (基于 Dmitry Vyukov 的有界 MPMC 算法)
// ---------------------------------------------------------------------------
template<typename T>
class LockFreeQueue {
public:
    explicit LockFreeQueue(size_t size) : buffer_(size), mask_(size - 1) {
        // 要求 size 必须是 2 的幂，以便使用位运算优化取模
        if ((size & (size - 1)) != 0) {
            throw std::invalid_argument("Queue size must be a power of 2");
        }

        // 初始化序列号：0, 1, 2...
        for (size_t i = 0; i < size; ++i) {
            buffer_[i].sequence.store(i, std::memory_order_relaxed);
        }
        enqueue_pos_.store(0, std::memory_order_relaxed);
        dequeue_pos_.store(0, std::memory_order_relaxed);
    }

    // 生产者：入队
    bool enqueue(T&& data) {
        // C++20 atomic::wait 唤醒机制所需
        // 这里简化为自旋+yield，更复杂的实现可结合 wait/notify

        size_t pos = enqueue_pos_.load(std::memory_order_relaxed);
        while (true) {
            Cell& cell = buffer_[pos & mask_];
            size_t seq = cell.sequence.load(std::memory_order_acquire);
            intptr_t dif = (intptr_t)seq - (intptr_t)pos;

            if (dif == 0) {
                // 序列号匹配，尝试抢占位置
                if (enqueue_pos_.compare_exchange_weak(pos, pos + 1, std::memory_order_relaxed)) {
                    // 抢占成功，写入数据
                    cell.data = std::move(data);
                    // 更新序列号，标记为"可读" (seq + 1)
                    cell.sequence.store(pos + 1, std::memory_order_release);
                    return true;
                }
            } else if (dif < 0) {
```

```
                // 队列已满
                return false;
            } else {
                // 此时 pos 已经被其他线程抢先了，重新加载
                pos = enqueue_pos_.load(std::memory_order_relaxed);
            }
        }
    }

    // 消费者：出队
    bool dequeue(T& data) {
        size_t pos = dequeue_pos_.load(std::memory_order_relaxed);
        while (true) {
            Cell& cell = buffer_[pos & mask_];
            size_t seq = cell.sequence.load(std::memory_order_acquire);
            intptr_t dif = (intptr_t)seq - (intptr_t)(pos + 1);

            if (dif == 0) {
                // 序列号匹配（pos + 1），表示有数据
                if (dequeue_pos_.compare_exchange_weak(pos, pos + 1, std::memory_order_relaxed)) {
                    data = std::move(cell.data);
                    // 更新序列号，标记为"空闲"，且加上 buffer_size 表示下一轮
                    cell.sequence.store(pos + mask_ + 1, std::memory_order_release);
                    return true;
                }
            } else if (dif < 0) {
                // 队列为空
                return false;
            } else {
                pos = dequeue_pos_.load(std::memory_order_relaxed);
            }
        }
    }

private:
    struct Cell {
        std::atomic<size_t> sequence;
        T data;
    };

    // 解决伪共享：确保 head 和 tail 不在同一个缓存行
    // C++17 引入 hardware_destructive_interference_size
    static constexpr size_t cacheline_size = std::hardware_destructive_interference_size;

    std::vector<Cell> buffer_;
    const size_t mask_;

    alignas(cacheline_size) std::atomic<size_t> enqueue_pos_;
    alignas(cacheline_size) std::atomic<size_t> dequeue_pos_;
};

// ---------------------------------------------------------------------------
// 2. ThreadPool (支持 C++20 jthread 和 osyncstream)
```

```cpp
// ---------------------------------------------------------------------------
class ThreadPool {
public:
    // 队列大小建议设为 2 的幂，例如 1024
    explicit ThreadPool(size_t num_threads = std::thread::hardware_concurrency(), size_t queue_size = 1024)
        : queue_(queue_size)
    {
        for (size_t i = 0; i < num_threads; ++i) {
            // C++20: jthread 自动 join，stop_token 处理停止
            workers_.emplace_back([this](std::stop_token st) {
                std::function<void()> task;
                while (!st.stop_requested()) {
                    if (queue_.dequeue(task)) {
                        task();
                    } else {
                        // 队列为空时，让出 CPU 时间片，避免死循环烧 CPU
                        // 在超高性能场景可以使用 atomic::wait 替代 yield
                        std::this_thread::yield();
                    }
                }
            });
        }
    }

    ~ThreadPool() {
        // jthread 的析构函数会自动 request_stop() 并 join()
        // 我们不需要手动写析构逻辑，非常简洁
    }

    // 提交任务
    template<class F, class... Args>
    auto submit(F&& f, Args&&... args) -> std::future<std::invoke_result_t<F, Args...>> {
        using ReturnType = std::invoke_result_t<F, Args...>;

        auto task = std::make_shared<std::packaged_task<ReturnType()>>(
            std::bind(std::forward<F>(f), std::forward<Args>(args)...)
        );

        std::future<ReturnType> res = task->get_future();

        // 尝试入队，如果队列满了简单的策略是自旋重试，或者抛出异常
        // 这里为了演示简单，使用 while 重试
        while (!queue_.enqueue([task]() { (*task)(); })) {
            std::this_thread::yield();
        }

        return res;
    }

private:
    LockFreeQueue<std::function<void()>> queue_;
    std::vector<std::jthread> workers_;
};
```

```cpp

// ---------------------------------------------------------------------------
// 3. 测试与展示
// ---------------------------------------------------------------------------
int main() {
    ThreadPool pool(4);
    std::vector<std::future<int>> results;

    std::osyncstream(std::cout) << "Main: Submitting tasks...\n";

    for (int i = 0; i < 8; ++i) {
        results.emplace_back(
            pool.submit([i] {
                // C++20 osyncstream: 线程安全的输出，不会乱序
                std::osyncstream(std::cout) << "Task " << i
                                            << " running in thread "
                                            << std::this_thread::get_id() << "\n";
                // 模拟计算
                int sum = 0;
                for(int k=0; k<10000; ++k) sum += k;
                return i * i;
            })
        );
    }

    for (auto&& res : results) {
        std::osyncstream(std::cout) << "Result: " << res.get() << "\n";
    }

    return 0;
}
```